Knots as Cost–Accounted Rholang Processes: A Dowker–Thistlethwaite–Driven Encoding and Weak–Bisimulation Invariance

L. G. Meredith
F1R3FLY.io

July 22, 2026

Abstract

We revisit a 2005 encoding of knot diagrams as concurrent processes and rebuild it in core rholang (the reflective higher-order ρ-calculus with a built-in name-restriction operator) equipped with cost accounting. A crossing becomes a *gate*: the over-strand passes a signal straight through and produces a private *latch* token; the under-strand is a *join* on its incoming signal and that latch. Chirality is nothing more than which strand produces the latch and which joins on it. Driving the construction from a Dowker–Thistlethwaite (DT) code makes every arc a single-producer / single-consumer channel, so the arcs *are* the wires and the whole knot is a flat parallel composition of gates plus injected "current." Under cost accounting the old *ad hoc* demand for "enough current" becomes a solvency condition on a token ledger, and the surplus current becomes a metered, hence internal, quantity. We sketch a proof that the encoding, taken as a monoidal functor from tangles to processes modulo weak bisimulation, is invariant under the Reidemeister moves, and observe that the cost surplus of the type-I move is exactly the writhe.

1 Introduction

The original intuition [1] was to read a knot diagram as a signalling network. Each crossing is a small gate with two through-paths, one for the over-strand and one for the under-strand. A signal on the over-strand passes straight through; a signal on the under-strand must wait for a *latch* to open, and the latch is opened by the over-strand's own signal as it passes. Arcs of the diagram carry signals from one gate port to another. The encoding was shown to send the Reidemeister moves to weak bisimulations, making weak-bisimulation class a diagram invariant.

Two ingredients were left implicit. First, because a strand that is over at one crossing may be under at another, the latch dependencies chain, and a single circulating signal can deadlock at an under-strand whose latch has not yet been produced. The original design simply *posited* "enough current" to keep the network live. Second, the encoding was never driven from a machine-readable knot representation, so it was never mechanically instantiated.

This note repairs both points. We work in core rholang as implemented by the `f1r3node-rust` interpreter [2], using the built-in `new` (whose several desugarings we do not revisit), the join receipt written with `&`, and a cost semantics that debits a token from a located stack on each communication. "Enough current" becomes a solvency invariant of that ledger, and the leftover current becomes internal traffic that weak bisimulation is entitled to erase. We generate the process directly from a DT code, and we recast the invariance result functorially over tangles so that the equivalence being preserved is genuinely observable at the tangle boundary.

2 Core rholang with cost accounting

We use the fragment of rholang generated by

$$P, Q \;::=\; \mathbf{0} \;\mid\; x!(Q) \;\mid\; \textstyle\sum_i \texttt{for}(\vec{y}_i \leftarrow \vec{z}_i)\{P_i\} \;\mid\; P \mid Q \;\mid\; \texttt{new}\, x\, \texttt{in}\, \{P\} \;\mid\; *x,$$

with quoting $@P$ and dereferencing $*x$ realising reflection. A *join* receipt $\texttt{for}(y_1 \leftarrow z_1 \;\&\; \cdots \;\&\; y_k \leftarrow z_k)\{P\}$ fires only when a message is simultaneously available on every z_j, consuming one from each; the general receipt is a `;`-separated list of such &-joined groups. We rely only on single-channel receives and two-channel joins.

Cost. Fix a commutative monoid of costs $(\mathcal{C}, +, 0)$ with a phlogiston supply. The cost semantics attaches to each communication rule a debit: a rendezvous on a channel n consumes one token from the located stack at n (equivalently, one unit of phlogiston). A term is *solvent* for a given endowment if no maximal reduction sequence blocks for want of a token. Write $\mathrm{c}(P)$ for the multiset of debits incurred along a run of P to a normal form; for the terms below every run has the same c up to reordering, so c is well defined.

3 The crossing gate

A crossing has four ports: over-in *oi*, over-out *oo*, under-in *ui*, under-out *uo*, and a private latch *l*. The gate is

```
new l in {
    for (s <- oi) { oo!(*s) | l!(Nil) }      // over: pass through, produce latch
  | for (s <- ui & _ <- l) { uo!(*s) }       // under: JOIN on signal & latch
}
```

The over-strand is the unique producer on l; the under-strand is the unique joiner on l. Nothing else distinguishes the two strands, so a crossing and its mirror differ exactly by swapping producer and joiner. Because the join is atomic, there is no intermediate state in which the under-signal has been consumed while the forward is still stalled: the gate either commits both halves or does nothing, which removes the only awkward interleaving of the nested-receive formulation.

4 From DT codes to processes

A DT traversal of an n-crossing diagram visits $2n$ *passes*, numbered $1, \ldots, 2n$; each crossing is visited twice, receiving one odd and one even label. The DT code lists, for each odd label $1, 3, \ldots, 2n-1$ in order, the signed even label sharing that crossing. We use the sign convention: $d_i > 0$ iff the odd-labelled pass is the over-strand (so an alternating diagram has all-positive code).

Let arc a_k be the segment from pass k to pass $(k \bmod 2n) + 1$. Then pass p is *entered* by a_{p-1} (with $a_0 \equiv a_{2n}$) and *exited* by a_p. Two facts make the arcs serve as bare channels:

Proposition 1. *In the encoding of a knot diagram every arc a_k occurs exactly once as the output port of a gate and exactly once as the input port of a gate.*

Proof. Each pass contributes exactly one exit (a send on a_p) and one entry (a receive on a_{p-1}). The $2n$ passes are in bijection with the $2n$ arcs on each side, and a knot diagram is a single closed curve, so the exit/entry incidences are a perfect matching on arcs. □

```
// trefoil 3_1, DT [4,6,2]  (3 crossings, 6 arcs)
new a1, a2, a3, a4, a5, a6 in {
  new l in {   for (s <- a6) { a1!(*s) | l!(Nil) }
             | for (s <- a3 & _ <- l) { a4!(*s) } }   // over a6->a1, under a3->a4
  |
  new l in {   for (s <- a2) { a3!(*s) | l!(Nil) }
             | for (s <- a5 & _ <- l) { a6!(*s) } }   // over a2->a3, under a5->a6
  |
  new l in {   for (s <- a4) { a5!(*s) | l!(Nil) }
             | for (s <- a1 & _ <- l) { a2!(*s) } }   // over a4->a5, under a1->a2
  |
  a1!(Nil) | a2!(Nil) | a3!(Nil) | a4!(Nil) | a5!(Nil) | a6!(Nil)   // current
}
```

Figure 1: The trefoil, generated from its DT code. Each arc `a`k is produced once and consumed once, per Proposition 1.

Hence there are no forwarder processes: wiring is name-sharing, and each arc is one metered communication. A short rholang contract `DTtoKnot` and an even shorter Python transliteration emit the term; both accompany this note.

Definition 1 (Encoding)**.** Given a DT code $d = (d_1, \ldots, d_n)$, let $[\![d]\!]$ be the term `new` $a_1, \ldots, a_{2n}$ `in` $\{ \big(\|_i G_i \big) \mid I \}$, where G_i is the gate of crossing i with its ports computed as above and $I = \|_{k=1}^{2n} a_k!(\mathbf{0})$ is the current.

5 Current, solvency, and cost

Lemma 1 (Solvency)**.** *For the all-arcs current I of Definition 1, $[\![d]\!]$ is solvent and deadlock-free: every gate fires exactly once, and a full run consists of $2n$ primary rendezvous (n over-receives and n under-joins).*

Proof sketch. Each over-receive `for`$(s \leftarrow oi)$ has an injected token waiting on its over-in arc and fires, producing its latch and a forward on its over-out arc. Once every latch is present, each under-join has its injected under-in token and its latch, and fires. No receive is ever permanently starved, so there is no deadlock. The n over-receives and n under-joins are the primary events; the $2n$ forwarded sends land on arcs whose injected token was already consumed, so they are residual. □

The residue—one unconsumed token per arc under the all-arcs policy—is exactly the "surplus current" the original design tolerated. Under cost accounting it is a definite quantity rather than a hope. The *minimal* solvent endowment (the least current that avoids deadlock) is the genuine optimisation problem hiding here; it coincides with the phlogiston budget and we leave it open.

6 Tangles, boundaries, and the encoding functor

To make "weak bisimulation" meaningful we work with *open tangles*. A tangle T with p inputs and q outputs is a piece of diagram with $p+q$ boundary endpoints; composition glues outputs to inputs, and juxtaposition places tangles side by side. Tangles form a (braided, framed) monoidal category **Tang** whose morphisms modulo ambient isotopy rel boundary are generated by cups, caps, the two braidings, and the identity, subject to the Reidemeister relations [4].

The encoding of a tangle keeps its boundary arcs *free*: the p input arcs and q output arcs are not restricted, so they are observable channels. Let $\mathbf{Proc}_{\approx}$ be rholang processes-with-boundary modulo weak bisimulation $\approx$, where $\approx$ abstracts the internal (latch and forward) τ-steps but observes communication on free boundary names. Extend Definition 1 to open tangles by supplying local current on internal arcs only.

Definition 2. $[\![-]\!]\colon \mathbf{Tang} \to \mathbf{Proc}_{\approx}$ sends a tangle to its gate network with free boundary; composition maps to boundary-name identification and juxtaposition to parallel composition.

Lemma 2 (Boundary forwarder)**.** *For every tangle T, $[\![T]\!]$ is weakly bisimilar to the pure forwarder that copies each input boundary signal to the output boundary endpoint reached by following the strand through T. All crossing machinery is internal.*

Proof sketch. By Lemma 1 applied locally, each gate relays its incoming signals to its outgoing ports through a bounded run of τ-steps, consuming its private latch. Composing gates along internal arcs composes forwarders; the internal arcs and latches are restricted, hence their traffic is τ. What remains observable is precisely the input$\to$output relaying at the free boundary. □

Lemma 3 (Congruence)**.** *$\approx$ is a congruence for parallel composition and name restriction. Hence if $[\![T]\!] \approx [\![T']\!]$ for tangles with equal boundary, then $[\![D[T]]\!] \approx [\![D[T']]\!]$ in every diagram context $D[-]$.*

Proof. Standard for the ρ-calculus: $|$ and `new` preserve weak bisimulation. Boundary identification is a substitution of free names, which $\approx$ also respects. □

7 Weak-bisimulation invariance

Theorem 1 (Reidemeister invariance)**.** *If diagrams D and D' are related by a finite sequence of Reidemeister moves, then $[\![D]\!] \approx [\![D']\!]$. Consequently $[\![-]\!]$ descends to a map from knot types to weak-bisimulation classes.*

Proof sketch. By Lemma 3 it suffices to verify the three moves as local tangle relations; each relates two tangles with the same boundary.

Type I (twist). The kinked $1 \to 1$ tangle has a single self-crossing: one strand is both over and under, its two passes consecutive. The over-pass produces the latch that the same strand's under-pass immediately joins on, so the input signal reaches the output through two internal τ-steps. By Lemma 2 the kink and the bare wire are the same $1 \to 1$ forwarder, hence weakly bisimilar. The cost differs: the kink adds one over-receive and one under-join, so $\mathrm{c}(\text{kink}) = \mathrm{c}(\text{wire}) + 2$.

Type II (poke). In the removable configuration one strand A passes over the other strand B at both crossings. A forwards straight through, producing two latches; B's signal passes two under-joins, each gated by one of A's latches. The result is two independent forwarders $A\colon a \to a'$ and $B\colon b \to b'$, identical to the two parallel wires; weak bisimilarity again follows from Lemma 2. The cost delta is $+4$ (two over-receives, two under-joins), local to the two crossings.

Type III (slide). Both tangles have three strands and three crossings, and the move preserves, pairwise, which strand of each crossing is the over-strand, as well as the input$\to$output connectivity of the three strands. Only the *location* of the crossings changes. Thus the two encodings induce the same boundary forwarder and differ only in the internal τ-schedule, so they are weakly bisimilar. Here c is unchanged: both sides have three crossings, i.e. six primary rendezvous.

Weak bisimilarity of the local tangles lifts to $[\![D]\!] \approx [\![D']\!]$ by Lemma 3, and the Reidemeister moves generate ambient isotopy. □

Remark 1 (Cost counts the writhe)**.** Types II and III leave c invariant (canceling, resp. preserving), while type I changes it by +2 per twist. Hence on regular-isotopy classes the primary cost is constant, and the only variation across an ambient-isotopy class is $2\,\mathrm{w}(D)$: *the metered cost surplus of the encoding is twice the writhe.* If an unframed invariant is wanted, one normalises by this surplus, exactly as the Kauffman bracket is normalised by a writhe factor to yield the Jones polynomial. The encoding is thus a regular-isotopy (framed) invariant on the nose, with the framing read directly off the ledger.

8 The Perko pair

The sharpest classical test for any diagram-level construction is the *Perko pair*: the two reduced 10-crossing diagrams Perko A (10_{161}) and Perko B (formerly 10_{162}), listed as distinct knots from Little (1885) through Rolfsen (1976) until Perko proved them the same knot in 1974 [5, 6]. It is the smallest instance of two reduced diagrams of the same prime knot that a naive tabulation separated, and—this is the point that matters for us—the two diagrams have *different writhes*; the pair is the historical counterexample to Little's 1900 claim that the writhe of a reduced diagram is a knot invariant [7]. Perko A has DT code

$$[\,4,\ 12,\ -16,\ 14,\ -18,\ 2,\ 8,\ -20,\ -10,\ -6\,],$$

from which $[\![-]\!]$ produces a 10-gate, 20-arc network; every arc is a single producer/single consumer (Proposition 1), and a full run is 20 primary rendezvous. The generated term accompanies this note.

The pair probes the encoding from two sides, and it passes both.

Invariance: the theorem must *not* separate them. Perko A and Perko B are the same knot, hence connected by Reidemeister moves, so Theorem 1 gives $[\![\mathrm{A}]\!] \approx [\![\mathrm{B}]\!]$ as boundary forwarders, and after closing the boundary $[\![\mathrm{A}]\!]_{\mathrm{cl}} \approx [\![\mathrm{B}]\!]_{\mathrm{cl}}$ by the congruence of $\approx$ under restriction (Lemma 3). What makes the Perko pair famously hard for humans is that Perko's connecting sequence is not monotone in crossing number: it must pass through diagrams with *more* than ten crossings. Our proof is indifferent to this. It never tracks crossing number or any complexity measure along the connecting path; it factors the equivalence through the three local relations and lifts by congruence. The crossing-number-increasing detour that defeats a greedy simplifier is invisible to the argument, so the encoding is not fooled the way the tables were.

Framing: the cost *does* separate them, and should. Because the two diagrams have different writhes, the writhe remark gives

$$\mathrm{c}([\![\mathrm{A}]\!]) - \mathrm{c}([\![\mathrm{B}]\!]) \;=\; 2\big(\mathrm{w}(\mathrm{A}) - \mathrm{w}(\mathrm{B})\big) \;\neq\; 0.$$

Thus the encoding assigns the pair the *same* weak-bisimulation class (correctly: one knot) while recording *distinct* cost ledgers (correctly: two framings). The unframed invariant is recovered after the writhe normalisation of the remark; the framed, regular-isotopy reading keeps A and B apart exactly where they genuinely differ.

Why this is a confirmation, not a threat. The metered surplus of the encoding is twice the writhe—precisely the quantity Little's 1900 "theorem" wrongly asserted to be a knot invariant, and precisely the quantity the Perko pair refutes. Theorem 1 states the correct status of that quantity:

it is a regular-isotopy (framed) invariant, not an ambient-isotopy one. The Perko pair instantiates that very distinction at minimal crossing number. Far from embarrassing the construction, it shows the encoding cleaves the data at the right joint: knot type lands in the weak-bisimulation class, framing lands in the cost, and the two are cleanly separated. The atom of the phenomenon is already the type-I move—a kink on the unknot is the smallest pair of same-knot diagrams with different writhes, contributing the +2 our ledger charges; the Perko pair is the smallest place where the accumulated effect distinguishes two reduced diagrams of a nontrivial prime knot.

9 The Jones polynomial via resolution-contexts

The gate has a second life. Read dynamically it is a latch; read structurally it is a four-port *hole*, and the Kauffman bracket is exactly what one computes by plugging those holes.

Smoothings are contexts. A crossing tangle has four boundary ports. There are two planar ways to reconnect them without a crossing—the two *smoothings*. In process terms each is a pair of wires, i.e. a *context* into which the gate-hole is plugged. Writing the four ports as two on top and two on bottom, the two connectors are

$\mathbf{1} = \{\,\text{top-left–bottom-left, top-right–bottom-right}\,\}$ (two through-wires), $e = \{\,\text{top-left–top-right, bottom-le}$

Replacing every gate in $[\![D]\!]$ by one of its two connectors yields a process of pure wire: a disjoint union of closed loops. A closed loop is a τ-cycle with no free port; its scalar value is $\delta = -A^2 - A^{-2}$.

The state sum. A *state* s assigns a smoothing to each of the n crossings; let $a(s)$ and $b(s)$ count the A- and B-smoothings and $|s|$ the number of resulting loops. The bracket is the sum over context-assignments

$$\langle D \rangle \;=\; \sum_{s} A^{\,a(s)-b(s)}\;\delta^{\,|s|-1}, \qquad \delta = -A^2 - A^{-2},$$

where the loop count $|s|$ is read directly off the pure-wire process as its number of τ-cycles. This is precisely a sum over the resolution-contexts of the gate-holes, weighted by the choice and valued by the closed-loop structure of the result.

Temperley–Lieb as the algebra of resolution-contexts. On a single crossing the two connectors $\mathbf{1}$ and e are the two diagrams of the Temperley–Lieb algebra TL_2, and Kauffman's skein relation is the linear expansion

$$\text{crossing} \;\longmapsto\; A\,\mathbf{1} + A^{-1}e, \qquad e^2 = \delta\, e.$$

The relation $e^2 = \delta e$ is the process fact "a closed wire loop is worth δ": stacking two cup–caps traps one loop. Composition of tangles is boundary-name identification, juxtaposition is parallel composition, and the closure of a diagram is the Markov trace that evaluates each remaining loop to δ. Thus the bracket is the monoidal-linear functor

$$\langle - \rangle \colon \mathbf{Tang} \longrightarrow \mathbb{Z}[A^{\pm 1}]\text{-}\mathbf{Mod}, \qquad \text{factoring through } \mathrm{TL}(\delta),$$

uniquely determined by its value on the one generating crossing. In the context-labelled reading, the two smoothings are two context labels; the bracket sums over context-assignments to the holes, and closed-loop evaluation is the trace. This is the promised recasting: the Kauffman bracket *is* the linear functor on the multicategory of resolution-contexts, and $\mathrm{TL}(\delta)$ is the algebra those contexts generate.

Worked example: the trefoil. Present the trefoil as the closure of the 2-strand braid σ_1^3. Each σ_1 maps to $A\,\mathbf{1} + A^{-1}e$ in TL_2; cubing (using $e^2 = \delta e$) gives

$$(A\,\mathbf{1} + A^{-1}e)^3 \;=\; A^3\,\mathbf{1} \;+\; \left(A - A^{-3} + A^{-7}\right) e.$$

The trace closure sends $\mathbf{1} \mapsto \delta$ (two loops) and $e \mapsto 1$ (one loop), so

$$\langle 3_1 \rangle \;=\; A^3\delta + \left(A - A^{-3} + A^{-7}\right) \;=\; -A^5 - A^{-3} + A^{-7}.$$

From bracket to Jones, and the writhe again. The bracket is invariant under the type-II and type-III moves but scales by $-A^{\pm 3}$ under type-I. The normalisation

$$f(D) \;=\; (-A^3)^{-\,\mathrm{w}(D)}\,\langle D \rangle$$

is invariant under all three moves, and the Jones polynomial is $V_K(t) = f(D)\big|_{A=t^{-1/4}}$. For the trefoil, $\mathrm{w} = +3$ gives $f = A^{-4} + A^{-12} - A^{-16}$ and hence

$$V_{3_1}(t) \;=\; -t^4 + t^3 + t.$$

The normalisation exponent is the writhe—the very quantity our cost ledger records as $2\,\mathrm{w}$ (Section 8). So the bracket-to-Jones passage, which trades the framed (regular-isotopy) bracket for the ambient-isotopy Jones invariant, is the *same* writhe quotient as the passage from our regular-isotopy encoding to an ambient one. One writhe, two faces: on the state-sum side it is the $(-A^3)^{-\,\mathrm{w}}$ factor; on the process side it is the cost surplus. The two halves of this note—weak bisimulation with cost, and the Kauffman state sum—meet exactly at the writhe, and the gate's two readings keep them apart cleanly: the latch reading carries the dynamics and the current, while the hole reading, having no over/under, computes the bracket with no current at all.

10 Outlook

A spatial–behavioural classifier. Theorem 1 gives invariance; the converse, distinguishing non-isotopic knots, is carried by the closed encoding rather than the boundary forwarder. Closing a tangle into a knot restricts all boundary names, so all behaviour becomes internal and the discriminating content moves into the *spatial* structure of the term—the arrangement and nesting of gates and latches. The logic generated for this structure by the OSLF construction is adequate by a theorem of the generating algorithm [3]: two closed terms satisfy the same formulae iff they are bisimilar. The remaining question is purely one of *faithfulness*—whether the induced bisimulation is as fine as knot type—not of adequacy. We expect the spatial-behavioural logic to be a sharper practical classifier than Conway's notation, which is a naming scheme tuned to rational tangles rather than an intrinsic, isotopy-respecting invariant.

Minimal current. The least solvent endowment—the true phlogiston budget of a diagram—may itself be an isotopy invariant, and its behaviour under the Reidemeister moves is the natural next computation.

A Rholang source

The listing below is the accompanying file `knot_encoding.rho` in full: the hand-expanded trefoil (Part 1), the parametric `DTtoKnot` builder (Part 2), and the Perko A term of Section 8 (Part 3). It targets the `f1r3node-rust` rholang interpreter; every send and receive carries a payload (the interpreter rejects 0-arity), no receive is guarded, and no arithmetic appears in a match pattern.

```
// ============================================================================
//  Knots as rholang processes  --  F1R3FLY.io working code
//  Target: f1r3node-rust rholang-cli
//
//  Crossing gate (four ports oi,oo,ui,uo; private latch l):
//     over strand : receive on oi, forward on oo, PRODUCE the latch l
//     under strand: JOIN on (ui, l) with '&', then forward on uo
//  Chirality == which strand produces l (over) vs. joins on it (under).
//
//  A DT traversal makes every arc a single-producer / single-consumer
//  channel, so the arcs ARE the wires: no forwarder processes.
//  "Current" is one message per arc; it is solvent by construction and the
//  leftover forwarded tokens are the metered surplus.
//
//  NB on interpreter limits (rholang/README.md): no 0-arity send/receive
//  (we always carry Nil), no guarded receive patterns, match patterns are
//  not pre-evaluated (we never put arithmetic in a pattern).
// ============================================================================


// ----------------------------------------------------------------------------
//  Part 1.  The trefoil 3_1, DT code [4,6,2], expanded by hand.  Runnable.
//
//  passes 1..6, arc a_k = pass k -> pass (k+1 mod 6).  pass p: in a_{p-1}, out a_p.
//  alternating => odd pass is OVER at every crossing (all-positive DT).
//    crossing 1 {odd 1, even 4}: over a6->a1, under a3->a4
//    crossing 2 {odd 3, even 6}: over a2->a3, under a5->a6
//    crossing 3 {odd 5, even 2}: over a4->a5, under a1->a2
// ----------------------------------------------------------------------------
new a1, a2, a3, a4, a5, a6, stdout(`rho:io:stdout`) in {

  // crossing 1
  new l in {
      for (s <- a6) { a1!(*s) | l!(Nil) }
    | for (s <- a3 & _ <- l) { a4!(*s) }
  }
  |
  // crossing 2
  new l in {
      for (s <- a2) { a3!(*s) | l!(Nil) }
    | for (s <- a5 & _ <- l) { a6!(*s) }
  }
  |
  // crossing 3
  new l in {
      for (s <- a4) { a5!(*s) | l!(Nil) }
    | for (s <- a1 & _ <- l) { a2!(*s) }
  }
  |
  // current: one token per arc (solvent; residue = surplus)
  a1!(Nil) | a2!(Nil) | a3!(Nil) | a4!(Nil) | a5!(Nil) | a6!(Nil)
  |
  stdout!("trefoil encoding running")
}


// ============================================================================
//  Part 2.  Parametric builder:  DT code  ->  knot process.
//
//  DTtoKnot(dt, ret): dt is a list of signed even labels (one per crossing).
//  Builds the whole network from a fresh unforgeable token `knot`; arc k is
//  the reflected name  @[*knot, k].  Returns *knot on ret so a caller can
//  seed / observe the boundary if desired.
//
//  Convention:  dt_i > 0  =>  the ODD-labelled pass is the OVER strand.
//  (Flip the (over,under) assignment below for the opposite convention.)
// ============================================================================
```

```
new DTtoKnot, length in {

  // length of a list
  contract length(@xs, ret) = {
    match xs {
      []        => ret!(0)
      [_ ...t]  => { new r in { length!(t, *r) | for (@m <- r) { ret!(m + 1) } } }
    }
  } |

  contract DTtoKnot(@dt, ret) = {
    new knot, gates, inject, lenCh in {

      length!(dt, *lenCh) |
      for (@len <- lenCh) {
        // N = 2n arcs
        new go in {
          // ---- spawn one gate per crossing ----
          // gates(remaining, i, N): i is 1-based crossing index (odd label = 2i-1)
          contract gates(@rem, @i, @nn) = {
            match rem {
              [] => Nil
              [d ...t] => {
                // even label and over/under choice
                new eCh in {
                  ( if (d < 0) { eCh!(0 - d) } else { eCh!(d) } ) |
                  for (@e <- eCh) {
                    new pick in {
                      // pick!(overPass, underPass)
                      ( if (d > 0) { pick!(2 * i - 1, e) } else { pick!(e, 2 * i - 1) } ) |
                      for (@over, @under <- pick) {
                        new oiCh, uiCh in {
                          // in-arc(p) = N if p==1 else p-1 ; out-arc(p) = p
                          ( if (over == 1)  { oiCh!(nn) } else { oiCh!(over - 1) } ) |
                          ( if (under == 1) { uiCh!(nn) } else { uiCh!(under - 1) } ) |
                          for (@oi <- oiCh & @ui <- uiCh) {
                            // the gate itself
                            new l in {
                                for (s <- @[*knot, oi]) {
                                  @[*knot, over]!(*s) | l!(Nil)
                                }
                              | for (s <- @[*knot, ui] & _ <- l) {
                                  @[*knot, under]!(*s)
                                }
                            }
                          }
                        }
                      }
                    }
                  }
                } |
                gates!(t, i + 1, nn)
              }
            }
          } |

          // ---- inject current on every arc 1..N ----
          contract inject(@k, @nn) = {
            if (k <= nn) { @[*knot, k]!(Nil) | inject!(k + 1, nn) } else { Nil }
          } |

          gates!(dt, 1, 2 * len) |
          inject!(1, 2 * len) |
          ret!(*knot)
        }
      }
    }
  } |
```

```
  // Example: build the trefoil and the figure-eight
  new r1, r2 in {
    DTtoKnot!([4, 6, 2], *r1) |
    DTtoKnot!([4, 6, 8, 2], *r2) |
    for (_ <- r1) { Nil } |
    for (_ <- r2) { Nil }
  }
}


// ========================================================================
//  Part 3.  The Perko pair.  Perko A = 10_161, DT [4,12,-16,14,-18,2,8,-20,-10,-6]
//  (KnotInfo / Wikipedia).  Perko A and Perko B (formerly 10_162) are the SAME
//  knot with DIFFERENT writhes -- the classical refutation of Little's
//  writhe-invariance claim.  Our encoding gives them weakly-bisimilar boundary
//  forwarders (same knot) but cost ledgers differing by 2*(w_A - w_B).
//  Generated by dt_to_rho.py; standard DT sign convention.
// ========================================================================
// Perko A  10_161 (formerly the Perko pair 10_161/10_162): DT [4, 12, -16, 14, -18, 2, 8, -20, -10, -6]  (n
    =10 crossings, 20 arcs)
new a1, a2, a3, a4, a5, a6, a7, a8, a9, a10, a11, a12, a13, a14, a15, a16, a17, a18, a19, a20 in {
  new l in {
      for (s <- a20) { a1!(*s) | l!(Nil) }
    | for (s <- a3 & _ <- l) { a4!(*s) }
  }  // crossing 1: over a20->a1, under a3->a4
  |
  new l in {
      for (s <- a2) { a3!(*s) | l!(Nil) }
    | for (s <- a11 & _ <- l) { a12!(*s) }
  }  // crossing 2: over a2->a3, under a11->a12
  |
  new l in {
      for (s <- a15) { a16!(*s) | l!(Nil) }
    | for (s <- a4 & _ <- l) { a5!(*s) }
  }  // crossing 3: over a15->a16, under a4->a5
  |
  new l in {
      for (s <- a6) { a7!(*s) | l!(Nil) }
    | for (s <- a13 & _ <- l) { a14!(*s) }
  }  // crossing 4: over a6->a7, under a13->a14
  |
  new l in {
      for (s <- a17) { a18!(*s) | l!(Nil) }
    | for (s <- a8 & _ <- l) { a9!(*s) }
  }  // crossing 5: over a17->a18, under a8->a9
  |
  new l in {
      for (s <- a10) { a11!(*s) | l!(Nil) }
    | for (s <- a1 & _ <- l) { a2!(*s) }
  }  // crossing 6: over a10->a11, under a1->a2
  |
  new l in {
      for (s <- a12) { a13!(*s) | l!(Nil) }
    | for (s <- a7 & _ <- l) { a8!(*s) }
  }  // crossing 7: over a12->a13, under a7->a8
  |
  new l in {
      for (s <- a19) { a20!(*s) | l!(Nil) }
    | for (s <- a14 & _ <- l) { a15!(*s) }
  }  // crossing 8: over a19->a20, under a14->a15
  |
  new l in {
      for (s <- a9) { a10!(*s) | l!(Nil) }
    | for (s <- a16 & _ <- l) { a17!(*s) }
  }  // crossing 9: over a9->a10, under a16->a17
  |
  new l in {
```

```
      for (s <- a5) { a6!(*s) | l!(Nil) }
    | for (s <- a18 & _ <- l) { a19!(*s) }
  }  // crossing 10: over a5->a6, under a18->a19
  |
  // current (solvent by construction; residue = metered surplus)
  a1!(Nil)  | a2!(Nil)  | a3!(Nil)  | a4!(Nil)  | a5!(Nil)  | a6!(Nil)  | a7!(Nil)  | a8!(Nil)  | a9!(Nil)  |
      a10!(Nil)  | a11!(Nil)  | a12!(Nil)  | a13!(Nil)  | a14!(Nil)  | a15!(Nil)  | a16!(Nil)  | a17!(Nil)  |
      a18!(Nil)  | a19!(Nil)  | a20!(Nil)
}
```

References

[1] L. G. Meredith. *Knots as processes* (working notes, 2005).

[2] F1R3FLY.io. *f1r3node-rust*: a pure-Rust F1R3FLY node with a rholang interpreter. https://github.com/F1R3FLY-io/f1r3node-rust.

[3] Operational Semantics in Logical Form (OSLF): auto-generation of adequate Hennessy–Milner logics for graph-structured lambda theories.

[4] V. Turaev. *Operator invariants of tangles and R-matrices* (for the tangle category and its generators/relations).

[5] K. A. Perko, Jr. *On the classification of knots.* Proc. Amer. Math. Soc. **45** (1974), 262–266.

[6] C. N. Little. *Non-alternating ± knots.* Trans. Roy. Soc. Edinburgh **39** (1900), 771–778.

[7] *Perko pair.* Wikipedia; the two reduced diagrams have different writhes, refuting Little's writhe-invariance claim (a Tait conjecture).

[8] L. H. Kauffman. *State models and the Jones polynomial.* Topology **26** (1987), 395–407.

[9] W. B. R. Lickorish. *An Introduction to Knot Theory.* Graduate Texts in Mathematics **175**, Springer, 1997.